

Markov Decision Processes & Dynamic Programming

A good plan stays good from wherever you land

Jinkyoo Park · KAIST

The deepest crossing — from choosing once, to choosing at every state your choices create

— HANDOFF

The handoff — one choice becomes a sequence

Every lecture so far chose x once. From here, a decision reshapes the state the next decision faces.

Where we are — the deepest crossing

STAGES • CROSSING	static → dynamic ○ — ●	MODEL model-based ● — ○	AGENTS single agent ● — ○
-------------------	-------------------------------	--------------------------------	----------------------------------

	LINEAGE A • OR / DYNAMIC PROGRAMMING	LINEAGE B • CONTROL THEORY
MODEL-BASED (ORIGIN)	MDP & Dynamic Programming LECTURE 7	Optimal Control / Planning LECTURE 9
DATA-DRIVEN (EXTENSION)	Value-Based RL LECTURE 8	Policy-Based RL LECTURE 10

Six lectures have moved along **one** axis — from a world handed to us to a world learned from data. This one moves along the other, and the model comes *back*: P and R are given again. What changes is that **a decision now creates the situation the next decision must face**.

Lecture 1 asked for a point, $\min_x f(x)$. This lecture asks for a **rule**, because we cannot know in advance which states the future will hand us.

The single optimisation splits into time — and **that changes what an answer even is**.

Why a sequence is not many single choices

from the deck's opening example

THE CHOICE, WITH NUMBERS

Standing in your final undergraduate year, three actions:

ACTION	REWARD NOW
get a job	\$4,000 / month
go to graduate school	\$500 / month
start a company	\$0 / month

A one-step-greedy rule takes the job and never looks up.

WHY THAT IS THE WRONG READING

Graduate school pays 500 — and buys a **state**. From it a further action (publish X papers) opens a distribution over *researcher, professor, private company, officer, no job*.

The 500 was never the point. **The state it bought was.**

Actions are coupled **through the state**: what you do now changes the options and rewards you will have later. So an action cannot be scored by its own reward — only by **the whole future it unlocks**.

Which forces a new object. Instead of a number $f(x)$ attached to a decision, we need a number attached to a *situation* — the total future reward a good plan can extract from it — plus an equation tying each situation's number to its successors'. That equation is Bellman's, and it is the lecture.

Two inheritances — Lecture 1 stretched, Lecture 3 unrolled

LECTURE 1 (STATIC)	LECTURE 7 (DYNAMIC)	WHAT TIME ADDS
decision variable x	action a_t chosen at state s_t	a choice, repeated
objective $f(x)$	return $\sum_t \gamma^t r_{t+1}$	rewards accumulate
the world is fixed	$s_{t+1} \sim P(\cdot \mid s_t, a_t)$	actions reshape the future
optimum x^*	optimal policy $\pi^*(s)$	a rule, not a point
"what is best here?"	"what is best from here on? "	the value function $V^*(s)$

And the second parent. Lecture 3 closed on the decision network — belief, action and value in one object — and wrote its answer as $\text{EU}(a \mid o) = \sum_{s'} P(s' \mid o, a) U(s')$ with $a^* = \arg \max_a \text{EU}(a \mid o)$. It then said out loud that this is **a one-step Bellman backup, three lectures before there is a name for it**, and that nothing would change except that s' would acquire a successor. That debt falls due today: **this lecture is the influence diagram with the horizon released from one.**

Where the two lineages split

the source deck's own taxonomy, pp. 4–5

MODEL BASED — T AND R GIVEN

	FINITE ACTIONS	INFINITE ACTIONS
discrete time	MDP (Lec 7)	dynamic system (Lec 9)
continuous time	continuous-time MDP	continuous-time system

$$P(s_{t+1} \mid s_t, a_t) \cdot x_{t+1} = f(x_t, u_t), \dot{x}_t = f(x_t, u_t)$$

Read the columns. What separates this lecture from Lecture 9 is **the size of the action space**: a finite $\mathcal{A}$ lets you enumerate and take a `max`; an infinite one does not, and the model stops being a table P and becomes a function f . Read the rows downward and the same split repeats after the model is deleted — which is why value-based and policy-based RL are **not two methods but two ancestries**.
Lecture 9 is the mirror of this lecture, not its sequel. Lecture 8 sits directly below it; Lecture 10 directly below Lecture 9.

MODEL FREE — LEARNED FROM EXPERIENCE

	FINITE ACTIONS	INFINITE ACTIONS
discrete time	value-based RL (Lec 8)	policy-based RL (Lec 10)
continuous time	—	—

The source deck leaves the bottom row empty. So, still, does the field.

The roadmap — four questions



- **Q1 — Why can't we just be greedy?** Actions couple through the state, so we need a **value of being somewhere**, not of doing something.
- **Q2 — What must that value satisfy?** The **Bellman equation** — consistency first, then optimality.
- **Q3 — How do we solve it, given the model?** **Policy iteration** and **value iteration**, which turn out to be one algorithm on two schedules.
- **Q4 — Why does either converge?** The dance of **generalised policy iteration**, and the **contraction** that makes its fixed point unique.

— ACT 1

The arena, and the value of a state

Q1. A greedy rule scores an action by its own reward. Build the object that scores a *situation* instead.

The arena — a Markov decision process

Q1 Why not greedy?

Q2 What must value satisfy?

Q3 How do we solve it?

Q4 Why does it converge?

A finite MDP is the tuple $\langle \mathcal{S}, \mathcal{A}, T, R, \gamma \rangle$:

- states $s \in \mathcal{S}$ and actions $a \in \mathcal{A}$, both finite;
- a **transition function** $T(s, a, s') = P(S_{t+1} = s' \mid S_t = s, A_t = a) = P(s' \mid s, a)$ — the source deck's own gloss: **also called the model, or the dynamics**;
- a **reward function** $R(s, a, s')$, or its expectation $\mathbb{E}[r_t \mid S_t = s, A_t = a, S_{t+1} = s']$ if the reward is itself random;
- a **discount** $\gamma \in [0, 1)$, and a start state s_0 (plus terminals s_T for episodic tasks).

Stationary means T and R do not depend on t . That is what lets a chain of time slices collapse into a *single* recursive equation rather than one equation per step — and the two spellings $T(s, a, s')$ and $P(s' \mid s, a)$ name the same object. Lecture 8 will use the second, and delete it.

The goal is not a trajectory. It is **a policy** $\pi : \mathcal{S} \rightarrow \mathcal{A}$ that names the best action at every state.

The Markov property — and the licence it grants

$$P(S_{t+1} = s' \mid S_t, A_t, S_{t-1}, A_{t-1}, \dots, S_0, A_0) = P(S_{t+1} = s' \mid S_t, A_t)$$

Given the present state, the future and the past are independent. A state signal that retains all the relevant information is called Markov — the position and velocity of a projectile, the configuration of stones on a Go board.

The consequence is the one that matters, and it is not obvious:

$$\pi^*(s_t, s_{t-1}, \dots, s_0) = \pi^*(s_t)$$

The best rule that may consult the **entire history** is no better than the best rule that consults only the present state.

Without it, "the value of a state" would be meaningless — value would have to be indexed by every path that could have led there, and there are exponentially many. The Markov property is what makes a table with one row per state a sufficient answer. Lecture 3 arrived at the same condition from the other side: it is a missing-edge pattern in a Bayesian network over $S_1, \dots, S_T$.

The goal — return, and the discount

The agent maximises accumulated reward, not immediate reward. Two settings:

EPISODIC — THERE IS A LAST STEP

$$U_t = r_t + r_{t+1} + \dots + r_T = \sum_{k=0}^T r_{t+k}$$

A maze, a game of Go, a game of chess. The sum is finite because the episode is.

CONTINUING — THERE IS NOT

$$U_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

γ keeps the sum finite — and, as we will see in Act 4, keeps the *whole machine* well-posed.

-
- $\gamma = 0$ — **myopic**: only the immediate reward counts, and we are back to the greedy rule that failed;
 - $\gamma \rightarrow 1$ — **far-sighted**: distant reward counts almost as much as near reward.
-

A warning from the source deck worth repeating: **the reward signal communicates what you want achieved, never how**. Reward a chess engine for taking pieces and it will take pieces; it will not win. Every reward specification is a modelling decision of the kind Lecture 1 taught, made under the extra hazard that the optimiser has many steps in which to find the loophole.

A policy is a rule, not a point

$$\pi : \mathcal{S} \rightarrow \mathcal{A}, \quad \pi^* = \arg \max_{\pi} \mathbb{E}[U^{\pi}(s)] \quad \text{for all } s$$

Those three words are the whole distance from Lecture 1. We do not want a policy that is best *on average over some start distribution*; we want one that is simultaneously best **from every state at once**. It is not obvious that such a policy exists. That it does is the first gift of the Bellman equation.

A policy may be deterministic, $a^* = \pi^*(s)$, or stochastic, $p(a \mid s) = \pi^*(s, a)$. For a finite MDP with known T and R **a deterministic optimal policy always exists**, so we take π deterministic throughout. Stochastic policies return in Lecture 8, where randomness buys exploration, and in Lecture 13, where it buys unpredictability against an opponent.

The value of a state, and the value of an action

V^π — HOW GOOD IS IT TO *BE* HERE

$$V^\pi(s) = \mathbb{E}_\pi[U_t \mid S_t = s] = \mathbb{E}_\pi\left[\sum_{k \geq 0} \gamma^k r_{t+k} \mid S_t = s\right]$$

The expected return from following π starting at s .

Q^π — HOW GOOD IS IT TO *DO THIS* HERE

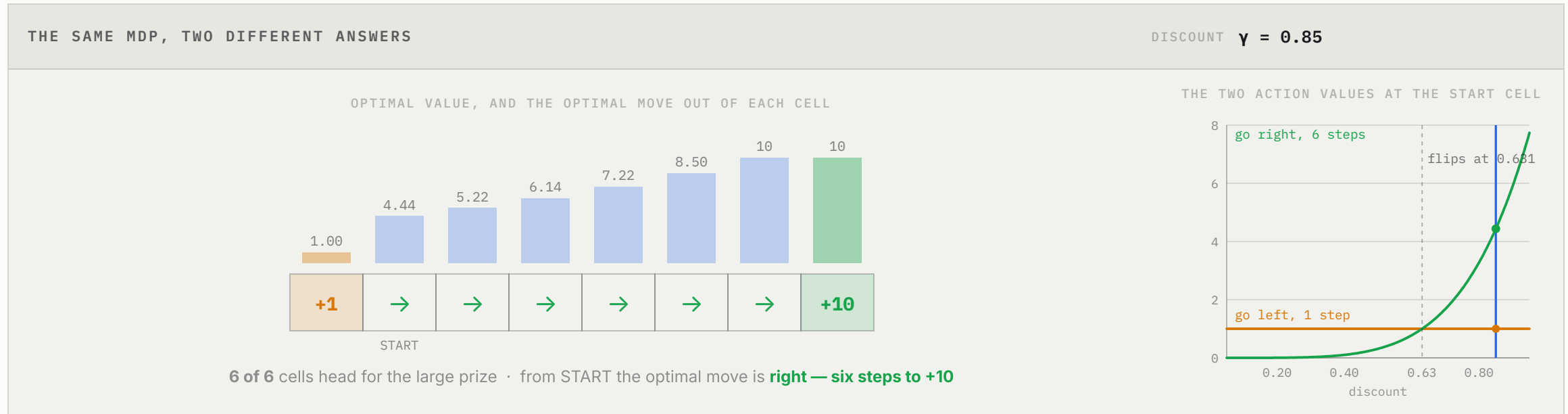
$$Q^\pi(s, a) = \mathbb{E}_\pi[U_t \mid S_t = s, A_t = a]$$

Take a now — whatever π would have said — then follow π forever after.

$\mathbb{E}_\pi$ is not an expectation over the policy; with π deterministic it is an expectation over **every stochastic transition the policy leads you through**. And both are defined *with respect to a policy*: there is no such thing as the value of a state, only the value of a state under a way of behaving.

This is exactly what the greedy rule lacked. A value scores a state by **the entire future a policy can extract from it**, not by one step's reward.

The discount is a decision, not bookkeeping



A corridor with a small prize one step to the left of START and a large prize six steps to the right. Turn γ and watch the arrows flip **one cell at a time, from the far end inwards**. The frontier is not a matter of taste — each cell has its own threshold, and they are exact: START switches at $\gamma = 10^{-1/5} \approx 0.631$, where $\gamma^5 \cdot 10$ overtakes 1; its neighbour at $10^{-1/3} \approx 0.464$; and the three cells nearest the large prize never point left at all. **Same MDP, same optimality principle, opposite answer.**

— ACT 2

The Bellman equation

Q2. The value of a state sums an infinite future. Turn that sum into a relation between neighbours.

Collapse the tree

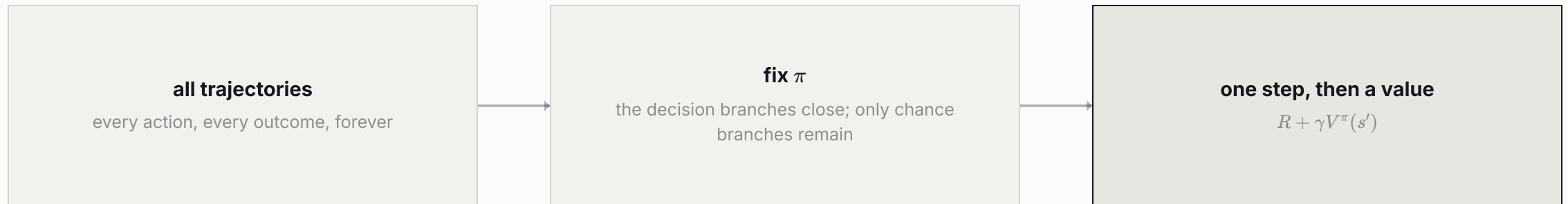
Q1 Why not greedy?

Q2 What must value satisfy?

Q3 How do we solve it?

Q4 Why does it converge?

Write out what $V^\pi(s)$ actually averages over. From s , every action; from each action, every successor; from each successor, every action again — a tree that never stops branching.



Fixing the policy prunes every decision branch. What remains is one layer of chance — and beyond it, a subtree that is *itself* a value function, because the Markov property says the future from s' depends on nothing but s' .

The infinite tree closes on itself. **That is the entire trick.**

The Bellman expectation equation

Peel one step off the return and let the Markov property close the loop:

$$V^\pi(s) = \mathbb{E}_\pi \left[\underbrace{r_t}_{\text{now}} + \gamma \underbrace{\sum_{k \geq 0} \gamma^k r_{t+1+k}}_{\text{the rest}} \mid S_t = s \right] = \sum_{s'} T(s, \pi(s), s') \left[R(s, \pi(s), s') + \gamma V^\pi(s') \right]$$

The value of a state = the expected immediate reward + the discounted value of where you land.

Read as a picture: a **state node** s leads to a **chance node** $(s, \pi(s))$, whose branches carry the weights $T(s, \pi(s), s')$ and end at successors holding $V^\pi(s')$. Every equation in this lecture is that diagram with one thing changed.

For a finite $\mathcal{S}$ this is **a system of $|\mathcal{S}|$ linear equations in $|\mathcal{S}|$ unknowns** — solvable in principle by matrix inversion, in $O(|\mathcal{S}|^3)$. It has turned "sum an infinite future" into "relate each state to its neighbours". That is the trade dynamic programming lives on.

Q and V — one lookahead apart

$$Q^\pi(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^\pi(s')], \quad V^\pi(s) = Q^\pi(s, \pi(s))$$

Q is the more general object: it keeps the first action free, then defers to π . Force $a = \pi(s)$ and it collapses back to V .

WHY WE WILL END UP STORING Q , NOT V

the source deck's sentence, kept whole

"The Q function effectively caches the results of all one-step-ahead search."

That cache is worthless while the model is free — with T in hand you can recompute the lookahead whenever you like. It becomes the whole ballgame the moment T is gone, because **the lookahead is precisely the thing you can no longer perform**. Lecture 8's first structural decision follows from this line and nothing else.

Bellman optimality — the max moves inside

For the *best* policy, the recursion takes a max. The step that makes it legal is worth seeing:

$$Q^*(s, a) = \max_{\pi} \mathbb{E}[R + \gamma V^{\pi}(s')] = \mathbb{E}[R + \gamma \max_{\pi} V^{\pi}(s')] = \mathbb{E}[R + \gamma V^*(s')]$$

The expectation runs over s' , which does not depend on π — so the maximisation slides past it and lands **at the successor**. Writing the two forms out:

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q^*(s', a')]$$

The two differ only in *where the max sits relative to the expectation*. V^* chooses first and then transitions; Q^* transitions first and then chooses at each outcome. Everything else on this slide is identical.

The principle of optimality, and the policy read off

BELLMAN'S PRINCIPLE OF OPTIMALITY

An optimal policy has the property that, whatever the first action and the state it leads to, the **remaining decisions must themselves be optimal from that state**.

Which is this lecture's thesis in formal dress: *a good plan stays good from wherever you land*. And it is what makes a daunting search over plans tractable — because it licenses solving the problem **one state at a time**. The optimal policy then falls straight out:

$$\pi^*(s) = \arg \max_a Q^*(s, a) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

The source deck's gloss: *any* greedy policy with respect to V^* is optimal, **because V^* already accounts for the reward consequences of all possible future behaviour**. Greed was never the enemy. Greed applied to the *immediate reward* fails; greed applied to **the right quantity** is exactly optimal. Act 1's question is now answered: we were not too greedy, we were greedy about the wrong number.

— ACT 3

Solving it with the model

Q3. The Bellman equations are conditions, not procedures. Turn them into iteration.

Two halves of one machine



Dynamic programming is, in the source deck's words, "a collection of algorithms that compute optimal policies **given a perfect model of the environment as a Markov decision process**". Its key idea is to use value functions to organise the search over policies. It has exactly two moves:

POLICY EVALUATION — MAKE V AGREE WITH Π

$$V_{k+1}(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R + \gamma V_k(s')]$$

Sweep the expectation backup until it stops changing.

POLICY IMPROVEMENT — MAKE Π GREEDY ON V

$$\pi'(s) = \arg \max_a \sum_{s'} T(s, a, s') [R + \gamma V^\pi(s')]$$

One arg max over a finite set, per state.

Set the right-hand box beside Lecture 1's $x^* = \arg \max_x f(x)$ — the source deck does exactly that, on one slide. Policy improvement *is* Lecture 1's arg max, executed once per state over $|\mathcal{A}|$ candidates instead of over a continuum. What is new is not the optimisation. It is **the quantity being optimised**.

Policy evaluation — sweeping the backup

```

Initialise  $V_0(s) \leftarrow 0$  for every  $s \in S$ 
Repeat for  $k = 0, 1, 2, \dots$ 
  for each state  $s$ :
     $V_{k+1}(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [ R(s, \pi(s), s') + \gamma V_k(s') ]$ 
until  $\max_s |V_{k+1}(s) - V_k(s)| \leq \epsilon$ 

```

This is a **full backup**: each sweep backs up the value of **every state, from every successor**, once. Nothing is sampled, nothing is skipped. Hold that phrase — Lecture 8's central move is to replace it with a *sample* backup from one successor.

TWO-ARRAY UPDATE

Read from V_k , write into V_{k+1} . The mathematically clean version: the sweep order cannot matter.

IN-PLACE UPDATE

One array. A state updated early in the sweep is read by states updated later, so **fresh information travels within a single sweep**. Usually faster, and half the memory.

The in-place variant is the first crack in the idea that DP must proceed in lockstep. Act 4 widens it into a hole.

The gridworld — the source deck's worked example

4×4, undiscounted, reward -1 on every move

THE MDP

- $\mathcal{S} = \{1, \dots, 14\}$ plus two shaded **terminal** corners;
- $\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$, deterministic;
- a move off the grid **leaves the state unchanged** — so $T(7, 7, \rightarrow) = 1$;
- $R = -1$ on *every* transition, and $\gamma = 1$.

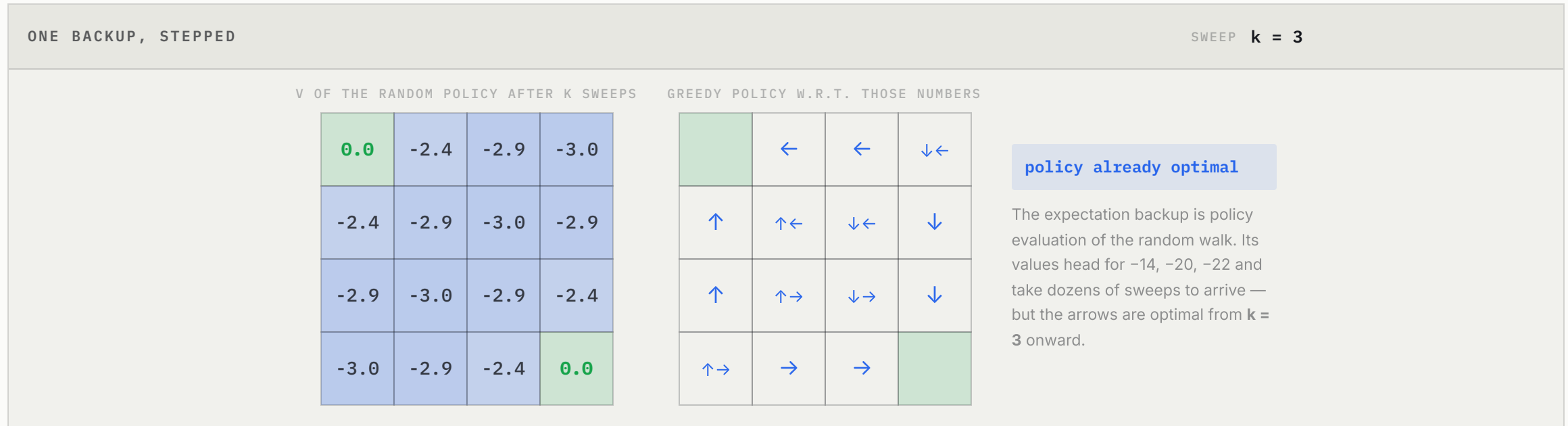
THE QUESTION

Under the **equiprobable random policy** — all four actions equally likely — what is V^π ?

Reward -1 everywhere means $-V^\pi(s)$ is simply **the expected number of steps a random walker takes to fall into a corner**.

$\gamma = 1$ is admissible here only because the task is *episodic* and every policy reaches a corner with probability one; with a policy that never terminates, undiscounted evaluation simply runs to $-\infty$. Both corners are terminal, so value propagates inward from *two* sources at once. Watch the next slide's $k = 1$ frame: after one sweep every non-terminal state reads exactly -1 , because a step costs 1 and everything it can reach is still worth 0. The information has not moved yet; after that it moves one ring per sweep.

Watching the value propagate



The deck's own figure, stepped. **Expectation backup** reproduces its printed table — -1.0, then -1.75 (which the printed figure truncates to -1.7), then -2.4, and at last -14, -20, -22. Switch to the **max backup** and the same sweep becomes value iteration: it settles in three sweeps at V^* = minus the distance to the nearest corner. The arrows are the greedy policy with respect to the numbers beside them, **recomputed every sweep**.

The policy is optimal long before the value is

The source deck draws a blue cross through the last two rows of its own figure. It is the most useful thing on the slide.

WHAT THE NUMBERS DO

At $k = 3$ the values read $-2.4, -2.9, -3.0$. They are heading for $-14, -20, -22$ and will take dozens more sweeps to get there.

WHAT THE POLICY DOES

At $k = 3$ the greedy policy is **already optimal, at every state**, and never changes again. At $k = 2$ it is not.

Evaluating to convergence was **wasted work**. The $\arg \max$ only needs the values *ordered* correctly, not converged.

This is the whole licence for truncating the inner loop, and everything in the rest of this act follows from it. It is also a warning: "the policy stopped changing" is a weaker certificate than "the value converged", and only the second one bounds how suboptimal you might still be.

Policy improvement — greedy on your own value

Given V^π , ask at each state whether a *single-step* deviation would pay:

$$\pi'(s) = \arg \max_a Q^\pi(s, a) \implies Q^\pi(s, \pi'(s)) \geq Q^\pi(s, \pi(s)) = V^\pi(s)$$

That inequality is cheap — it says *one* deviation, then back to π , is no worse. The theorem is the leap from there to *always*:

POLICY IMPROVEMENT THEOREM

If $\pi'(s) = \arg \max_a Q^\pi(s, a)$ then $V^{\pi'}(s) \geq V^\pi(s)$ for every s , with equality only when π is already greedy with respect to its own value — **which is exactly the Bellman optimality condition**.

The proof (Appendix, Backup 3) is a telescope: substitute the inequality, expand one step, substitute again. Each pass converts one more step from π to π' and the inequality never turns round. Two consequences. Improvement **can never hurt**; and since a finite MDP has finitely many deterministic policies and each round strictly improves, **policy iteration terminates exactly**.

Policy iteration, and its bottleneck

$$\pi_0 \xrightarrow{\text{PE}} V^{\pi_0} \xrightarrow{\text{PI}} \pi_1 \xrightarrow{\text{PE}} V^{\pi_1} \xrightarrow{\text{PI}} \pi_2 \longrightarrow \dots \longrightarrow \pi^*$$

Alternate the two moves until the policy stops changing. Each round gives an *exact* value for an *exact* policy, and the count of rounds is small.

THE BOTTLENECK

Every arrow marked PE hides a whole iterative solve — many sweeps over the entire state set. The source deck flags it bluntly: **policy evaluation is slow to converge.**

THE MITIGATION

Each evaluation **warm-starts from the previous policy's value**, and consecutive policies are similar, so later solves are far cheaper than the first.

And the previous slide has already told us the deeper answer: since the greedy step only needs the values *ranked*, running the inner loop to convergence is buying precision we throw away.

Value iteration — truncate the evaluation

Stop policy evaluation after **one** sweep, and fuse it with the improvement sweep that follows:

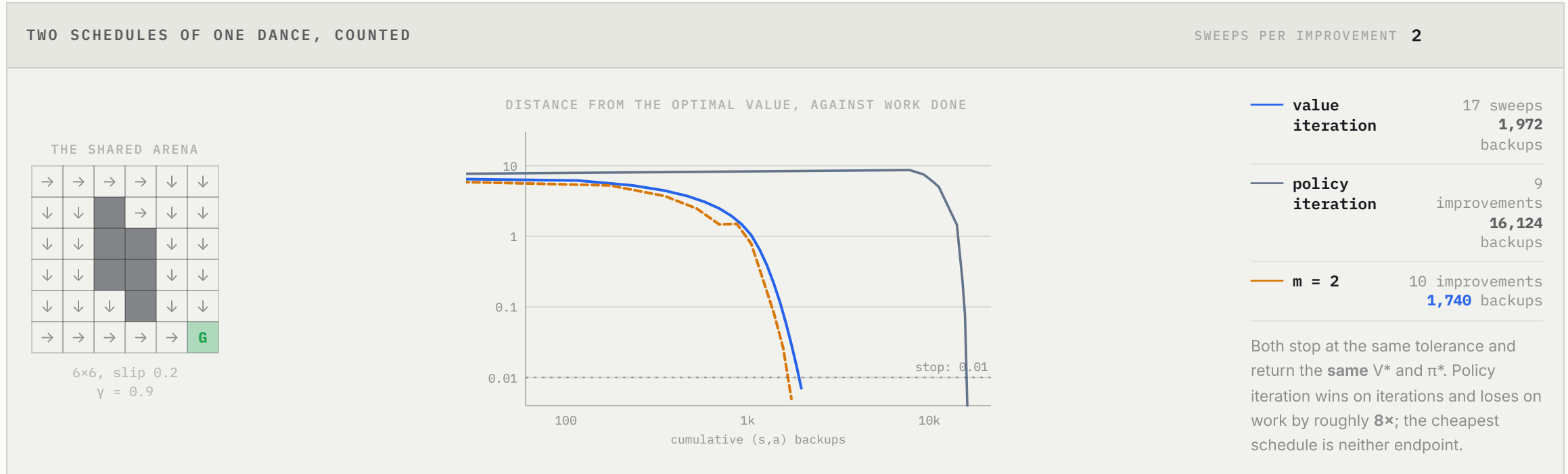
$$\underbrace{V_{k+1}(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R + \gamma V_k(s')]}_{\text{one evaluation sweep}} + \underbrace{\pi'(s) = \arg \max_a \sum_{s'} T(s, a, s') [R + \gamma V_{k+1}(s')]}_{\text{one improvement sweep}}$$

$$\Rightarrow V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

Or, with no derivation at all: it is **the Bellman optimality equation turned into an assignment**. The policy is never stored — it is read off once at the end, $\pi^*(s) = \arg \max_a \sum_{s'} T[R + \gamma V^*(s')]$.

Keep that boxed line in view for the rest of the course. Lecture 8's Q-learning is **this exact line** with the model-weighted sum $\sum_{s'} T(s, a, s') [\cdot]$ replaced by a single sampled successor, and the table replaced by a network. Nothing else about it changes.

Two schedules, counted



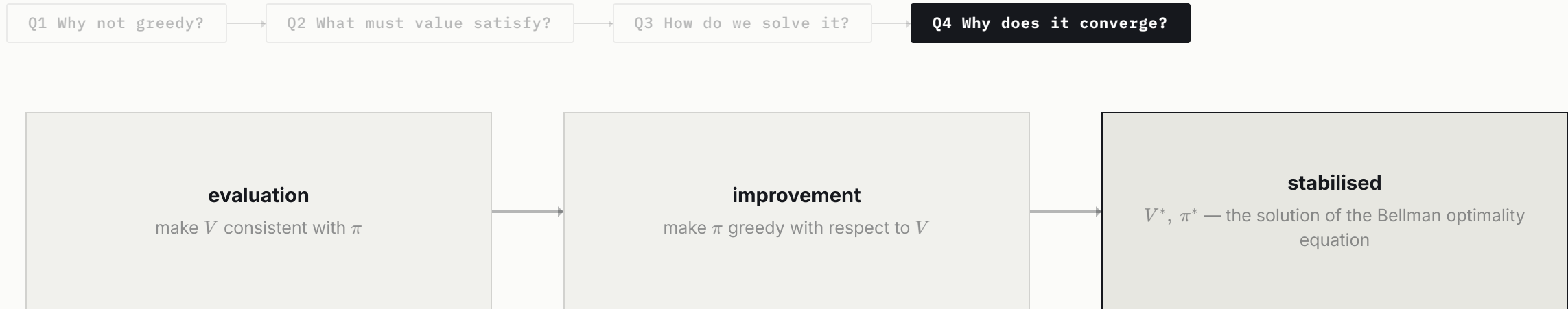
One MDP, one stopping rule, both algorithms — and a dial for the number of evaluation sweeps per improvement. At $m = \infty$ the schedule *is* policy iteration; drive m down and it slides continuously toward value iteration. Policy iteration reaches π^* in **9 improvements** but spends **16,124 backups** getting there; value iteration needs **17 sweeps** and only **1,972**. Same V^* , same π^* , an eightfold difference in work — and the cheapest schedule, at $m = 2$, is **neither endpoint**.

— ACT 4

Why it works

Q4. Two processes that keep undoing each other, and an answer they nevertheless agree on.

Generalised policy iteration — the dance behind both



Each move breaks the other. Making π greedy makes V wrong for the new π ; re-evaluating makes π no longer greedy. The source deck draws this as two lines in the plane of (V, π) — the locus $V = V^\pi$ and the locus $\pi = \text{greedy}(V)$ — with the algorithm **zigzagging between them in ever shorter hops**.

The two lines meet in **exactly one place**, and that meeting point is the solution of the Bellman optimality equation. Policy iteration and value iteration are two **schedules** of one dance, not two algorithms.

Why the meeting point is unique — the contraction

Define the **Bellman optimality operator** $\mathcal{T}$ on value functions:

$$(\mathcal{T}V)(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]$$

Value iteration is nothing but $V_{k+1} = \mathcal{T}V_k$. And $\mathcal{T}$ is a **γ -contraction in the sup-norm**:

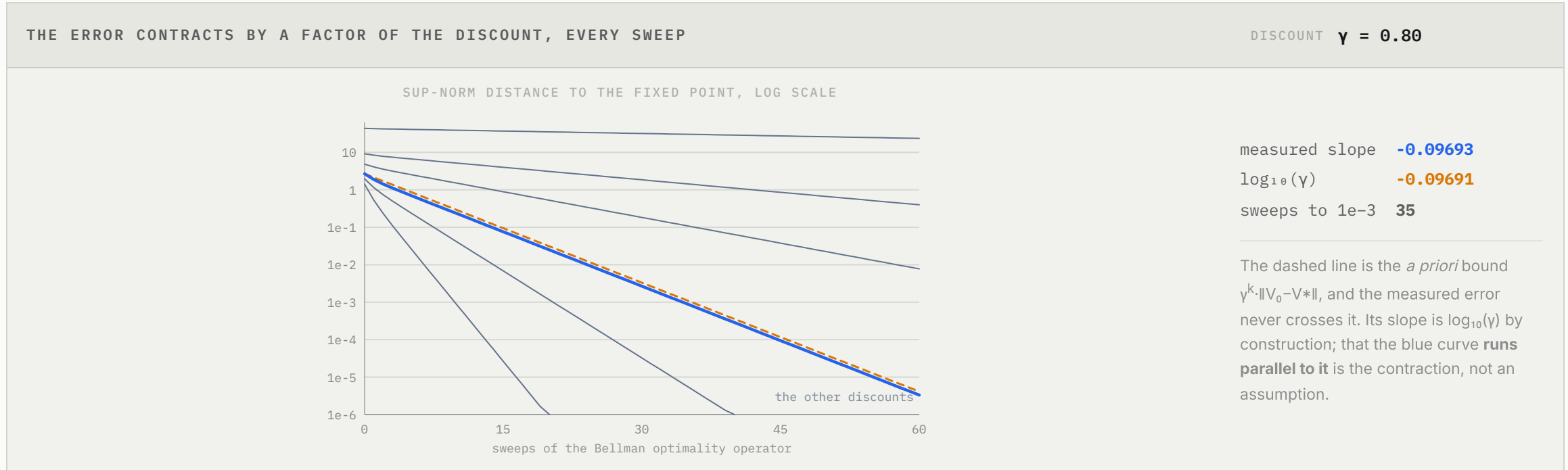
$$\|\mathcal{T}V - \mathcal{T}V'\|_\infty \leq \gamma \|V - V'\|_\infty$$

Two value functions disagreeing by δ anywhere are pushed to within $\gamma\delta$. By the Banach fixed-point theorem, $\gamma < 1$ then buys three things at once:

- a **unique** fixed point V^* — one answer, not a family;
- **convergence from any** V_0 , with no condition on the initialisation;
- a **rate**: $\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty$.

γ is not a modelling afterthought bolted on to keep a sum finite. It is **the number that makes the whole machine well-posed**, and it is also its convergence rate.

The rate is the discount



$\|V_k - V^*\|_\infty$ against sweep number, on a log axis. The claim of the last slide is that this is a **straight line of slope $\log \gamma$** — and it is: the measured slope tracks $\log_{10} \gamma$ to within 5×10^{-5} at every setting of the dial, and the curve never crosses the dashed *a priori* bound $\gamma^k \|V_0 - V^*\|$. Move the dial and watch it tilt. Reaching an error of 10^{-3} takes 10 sweeps at $\gamma = 0.5$ and more than 60 at $\gamma = 0.9$: **far-sightedness is not free, and γ is the invoice.**

Order does not matter either — asynchronous DP

Every sweep so far updated all states in lockstep. Nothing required that.

- **In-place:** overwrite $V(s)$ immediately, and reuse it within the same sweep.
- **Asynchronous:** back states up in **any order whatsoever**, using whatever values of the others happen to be available — repeating some, skipping others. Provided every state is updated **infinitely often**, $V \rightarrow V^*$ still holds.

The motivation is brute size. Backgammon has about 10^{20} states and Go has $3^{19 \times 19}$; a single full sweep over either will not finish, and conventional DP **improves nothing until an entire sweep completes**.

THE SOURCE DECK'S OWN ARROW FORWARD, EXCLAMATION MARKS INCLUDED

"Make it easier to intermix computation with real-time interaction: to solve a given MDP, we can run the iterative DP algorithm at the same time that an agent is actually experiencing the MDP (**Reinforcement Learning !!!!**)"

Sampled RL is asynchronous DP driven by whichever states the trajectory happens to visit. That is not an analogy — the "visit every state infinitely often" condition in Lecture 8's convergence theorem **is this slide's condition**, inherited unchanged.

Three things we have not paid for

The machine is complete and provably correct. Every part of it borrows something.

WHAT VALUE ITERATION DOES	WHAT IT NEEDS	WHAT HAPPENS WITHOUT IT
$\sum_{s'} T(s, a, s')[\dots]$	the transition model T	the expectation cannot be evaluated
$\arg \max_a \sum_{s'} T[R + \gamma V(s')]$	T again, for the lookahead	V alone cannot name an action
one entry $V(s)$ per state	a table the size of $\mathcal{S}$	no storage, and no generalisation to unseen states

Three debts, and the next lecture defaults on all three. Notice they are not independent: the first two are the same debt — **the model** — appearing once in evaluation and once in improvement. The third is why the last act of Lecture 8 exists at all.

— CLOSING

Closing

The equation is proved, the solvers are exact, and everything rested on owning the world.

Where we are — one parent established

	LINEAGE A • OR / DYNAMIC PROGRAMMING	LINEAGE B • CONTROL THEORY
MODEL-BASED (ORIGIN)	MDP & Dynamic Programming LECTURE 7	Optimal Control / Planning LECTURE 9
DATA-DRIVEN (EXTENSION)	Value-Based RL LECTURE 8	Policy-Based RL LECTURE 10

We hold the Bellman equation, two exact solvers, and a convergence guarantee — and every line of it leaned on **knowing T and R** . Two roads lead out, and the course takes both.

DOWN — LECTURE 8

Keep the equation, **delete the model**. Replace the expectation with a sample, V with Q , the table with a network. Value-based reinforcement learning: this lecture with (T, R) removed.

RIGHT — LECTURE 9

The *other* parent. Control theory solves the same dynamic problem with **an infinite action space and a dynamics function f** rather than a finite $\mathcal{A}$ and a kernel P — and Lecture 10 will delete f exactly as Lecture 8 deletes (T, R) .

What Lecture 8 receives

This lecture hands on [the Bellman equation, which still needs the model](#). In the form the next lecture opens with:

$$Q^*(s, a) = \sum_{s'} P(s' | s, a) \left[R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right]$$

And the objects that solve it, each with a sampled successor waiting:

WHAT WE HAND OVER	BUILT IN	WHAT LECTURE 8 DOES TO IT
the expectation $\sum_{s'} P(s' s, a)[\dots]$	Act 2	one sampled transition (s, a, r, s')
policy evaluation, solved with T, R	Act 3	MC returns, or a TD bootstrap
greedy on V : $\arg \max_a \sum_{s'} P[\dots]$	Acts 2–3	greedy on Q — the cache, now essential
value iteration	Act 3	Q-learning: the same max, sampled
the full backup, all successors	Act 3	a sample backup, one successor
a table, exact, one entry per state	Acts 3–4	a function $\hat{Q}(s, a; w)$

Column three is the whole of Lecture 8. Not one row changes the *principle* — only [what we may look up](#).

A sequence of decisions seemed to need a plan for
every possible future.

Bellman showed it needs only a **value for every state** — because an optimal plan, by its nature, stays optimal from wherever you happen to land.

Questions?

Value, defined recursively, turns an intractable search over plans into local backups over states. It is the founding move of operations-research decision making, the equation Lecture 8 keeps after throwing the model away, and — read one column to the right — the problem Lecture 9 will solve all over again in a different language.

— APPENDIX

Backup slides

The derivations, kept out of the narrative.

Backup 1 — the Bellman equation, derived

Start from the definition and peel off one step, using linearity of expectation and the Markov property:

$$\begin{aligned}
 V^\pi(s) &= \mathbb{E}_\pi \left[\sum_{k \geq 0} \gamma^k r_{t+k} \mid S_t = s \right] = \mathbb{E}_\pi \left[r_t + \gamma \sum_{k \geq 0} \gamma^k r_{t+1+k} \mid S_t = s \right] \\
 &= \sum_{s'} T(s, \pi(s), s') \left[R(s, \pi(s), s') + \gamma \mathbb{E}_\pi \left[\sum_{k \geq 0} \gamma^k r_{t+1+k} \mid S_{t+1} = s' \right] \right] \\
 &= \sum_{s'} T(s, \pi(s), s') \left[R(s, \pi(s), s') + \gamma V^\pi(s') \right] \quad \blacksquare
 \end{aligned}$$

The third line is where the Markov property is spent: the inner expectation depends on the whole history only through s' , so it is $V^\pi(s')$.

Optimality form. Maximise over the first action and use the principle of optimality for the remainder: $\pi(s)$ becomes $\max_a$ and V^π becomes V^* . **Q form.** Condition on (s, a) before the transition instead of after it; the $\max$ then lands at the successor, $\max_{a'} Q^*(s', a')$, which is why V^* maxes before the expectation and Q^* after.

Backup 2 — the contraction proof

Claim. $\|\mathcal{T}V - \mathcal{T}V'\|_\infty \leq \gamma\|V - V'\|_\infty$, where $(\mathcal{T}V)(s) = \max_a \sum_{s'} T(s, a, s') [R + \gamma V(s')]$.

For any state s , using $|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$:

$$\begin{aligned} |(\mathcal{T}V)(s) - (\mathcal{T}V')(s)| &\leq \max_a \left| \sum_{s'} T(s, a, s') \gamma [V(s') - V'(s')] \right| \\ &\leq \gamma \max_a \sum_{s'} T(s, a, s') |V(s') - V'(s')| \leq \gamma \|V - V'\|_\infty \end{aligned}$$

since $\sum_{s'} T(s, a, s') = 1$. Taking the maximum over s gives the claim. ■

Consequence (Banach). For $\gamma < 1$, $\mathcal{T}$ has a unique fixed point V^* , and $V_{k+1} = \mathcal{T}V_k$ converges to it from any V_0 with $\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty$. The max inequality is the only step that needs care, and it is where $|\mathcal{A}| < \infty$ is used — Lecture 9 works in a setting where that maximum is over a continuum. The same argument applied to a *sampled* operator underlies Q-learning's convergence (Lecture 8, Backup 2).

Backup 3 — policy improvement never hurts

Theorem. Let $\pi'(s) = \arg \max_a Q^\pi(s, a)$. Then $V^{\pi'}(s) \geq V^\pi(s)$ for all s .

By construction $Q^\pi(s, \pi'(s)) = \max_a Q^\pi(s, a) \geq Q^\pi(s, \pi(s)) = V^\pi(s)$. Now telescope, substituting that same inequality at each successor:

$$\begin{aligned} V^\pi(s) &\leq Q^\pi(s, \pi'(s)) = \mathbb{E}_{\pi'}[r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t = s] \\ &\leq \mathbb{E}_{\pi'}[r_{t+1} + \gamma Q^\pi(s_{t+1}, \pi'(s_{t+1})) \mid s_t = s] = \mathbb{E}_{\pi'}[r_{t+1} + \gamma r_{t+2} + \gamma^2 V^\pi(s_{t+2}) \mid s_t = s] \\ &\leq \cdots \leq \mathbb{E}_{\pi'}[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots \mid s_t = s] = V^{\pi'}(s) \quad \blacksquare \end{aligned}$$

Each pass converts one more step of the trajectory from π to π' , and the inequality never turns round. **Why policy iteration terminates.** A finite MDP has finitely many deterministic policies, and each improvement is strict unless π is already greedy with respect to its own value — which is precisely the Bellman optimality condition. So π^* is reached in finitely many rounds, not merely in the limit.

Backup 4 — asynchronous and in-place DP

In-place value iteration. Keep one array; overwrite $V(s)$ immediately and let later states in the same sweep read the new number. It converges, and usually faster, because fresh information propagates within a sweep instead of waiting for the next one. The sweep order now matters for *speed* — never for the answer.

Asynchronous DP. Back states up in any order, repeating some and skipping others. As long as every state is updated infinitely often, $V \rightarrow V^*$. This decouples the computation from any rigid sweep and is what lets DP be *prioritised* — order the backups by how large their Bellman residual was last time (prioritised sweeping), and effort follows the states where the value is still wrong.

Why it matters downstream. Sampled RL is asynchronous DP whose update order is chosen for it by the environment: Q-learning backs up whatever (s, a) the trajectory delivers, in whatever order experience delivers them. The "every state infinitely often" condition here is the model-based ancestor of the exploration requirement in Lecture 8 — and the reason ϵ must not be allowed to reach zero too soon.